

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories - system processes and user processes. System processes are given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>
```

```
#define MAX 100
```

```
typedef struct q
```

```
{
    int id, arrival, burst, remaining, type;
```

```
    int completion, turnaround, waiting;
```

```
} Process;
```

```
typedef struct q
```

```
{
    int items[MAX];
```

```
    int front, rear;
```

```
} Queue;
```

```
void initQueue (Queue *q) {
```

```
    q->front = q->rear = -1;
```

```
}
```

```
int isEmpty (Queue *q) {
```

```
    return q->front == -1;
```

```
}
```

```
void enqueue (Queue *q, int val) {
```

```
    if (q->rear == MAX-1) return;
```

```
    if (q->front == -1) q->front = 0;
```

```
    q->items[++q->rear] = val;
```

```
}
```



```

int dequeue (Queue *q) {
    if (isEmpty(q)) return -1;
    int val = q->items[q->front];
    if (q->front == q->rear) {
        q->front = q->rear = -1;
    } else {
        q->front++;
    }
    return val;
}

```

```

void sort (Process p[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (p[i].arrival > p[j].arrival) {
                Process temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}

```

```

int main() {
    int n;
    Process p[100];
    Queue systemQ, userQ;
    initQueue (&systemQ);
    initQueue (&userQ);
    printf ("Enter number of processes: ");
    scanf ("%d", &n);
    for (int i = 0; i < n; i++) {
        p[i].arrival = ...;
        p[i].priority = ...;
    }
}

```



```

for (int i=0; i<n; i++) {
    printf("In Process no: %d\n", i);
    p[i].id = i;
    printf("Enter arrival time: ");
    scanf("%d", &p[i].burst);
    printf("Enter type (0=system, 1=user): ");
    scanf("%d", &p[i].type);
    p[i].remaining = p[i].burst;
}

```

```

sort(p, n);

```

```

int time = 0, completed = 0, i = 0;

```

```

int current = -1;

```

```

int gantt[1000];

```

```

int gindex = 0;

```

```

while (completed < n) {

```

```

    while (i < n && p[i].arrival < time) {

```

```

        if (p[i].type == 0)

```

```

            enqueue(&systemQ, i);

```

```

        else

```

```

            enqueue(&userQ, i);

```

```

        i++;
    }

```

```

}

```

```

if (current != -1) {

```

```

    if (p[current].type == 1 && !isEmpty(&systemQ)) {

```

```

        enqueue(&userQ, current);

```

```

        current = -1;
    }

```

```

}

```

```

}

```

```

if (current == -1) {

```

```

    if (!isEmpty(&systemQ))

```

```

        current = dequeue(&systemQ);

```

```

    else if (!isEmpty(&userQ))

```

```

        current = dequeue(&userQ);

```


else if (time > 0) {

gantt[gindex++] = -1;

time++;

continue;

}

}

gantt[gindex++] = p[current].id;

p[current].remaining--;

time++;

if (p[current].remaining == 0) {

p[current].completion = time;

p[current].turnaround = time - p[current].arrive;

p[current].waiting = p[current].turnaround - p[current].burst;

completed++;

current = -1;

}

}

printf("\n Gantt Chart: \n");

for (int i = 0; i < gindex; i++) {

if (gantt[i] == -1)

printf("Idle 1");

else

printf("P%d", gantt[i]);

}

printf("\n");

for (int i = 1; i <= gindex; i++) {

printf(" %d", i);

}

float totalWT = 0, totalTAT = 0;

printf("\n\n ID | Etype | EAT | BT | CT | BWT | TAT | n");


```

for (int i = 0; i < n; i++) {
    printf ("Process %d: arrival time %d, burst time %d, type %d\n", i+1, p[i].arrival, p[i].burst, p[i].type);
    p[i].id = i+1;
    if (p[i].type == 0) {
        p[i].type = "System";
    } else {
        p[i].type = "User";
    }
    p[i].arrival = p[i].arrival;
    p[i].burst = p[i].burst;
    p[i].completion = 0;
    p[i].waiting = 0;
    p[i].turnaround = 0;

    totalWT += p[i].waiting;
    totalTAT += p[i].turnaround;

    printf ("Process %d: completion time %d, turnaround time %d, waiting time %d\n", i+1, p[i].completion, p[i].turnaround, p[i].waiting);
}

printf ("Average Waiting Time: %.2f\n", totalWT/n);
printf ("Average Turnaround Time: %.2f\n", totalTAT/n);
return 0;
}

```

O/P.

Enter number of Processes: 4

Process: 0

Enter arrival time: 0

Enter burst time: 5

Enter type (0 = System, 1 = User): 0

Process 1

Enter arrival time: 2

Enter burst time: 3

Enter type (0 = System, 1 = User): 2

Process 2

Enter arrival time: 1

Enter burst time: 4

Enter type (0 = System, 1 = User): 0

Process: 3 (P0, P1, P2) with length of 5, 4, 3

Enter arrival time: 3

Enter burst time: 2

Enter type (0=System, 1=User): 1

Grant chart:

1 P0 1 P0 1 P0 1 P0 1 P2 1 P2 1 P2 (P1) 1 P1 1 P1 1 P3 1 P3
 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14

ID	Type	AT	BT	CT	WT	TAT
0	System	0	5	5	0	5
2	System	1	4	9	4	8
1	User	2	3	12	7	10
3	User	3	2	14	9	11

24/3
 24/3